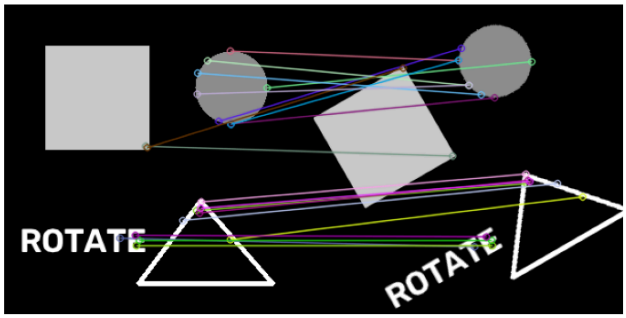
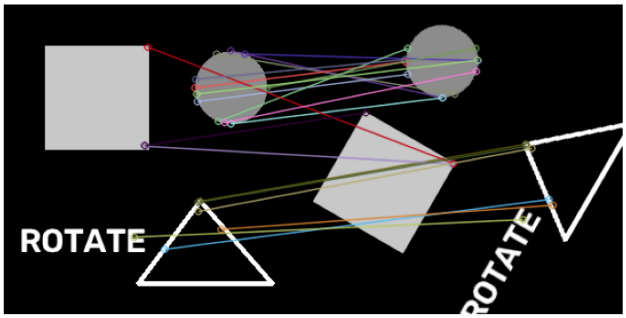
```
# ================================================================================
# QUESTION 5: Rotation Impact on Feature Matching
# Description: Generates a reference image, rotates it at distinct angles, and
#              evaluates ORB feature descriptor matching performance.
# ================================================================================

import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Generate Input Reference Image directly inside the script
img_15 = np.zeros((300, 300), dtype=np.uint8)
cv2.rectangle(img_15, (40, 40), (140, 140), 200, -1)
cv2.circle(img_15, (220, 80), 35, 140, -1)
cv2.polylines(img_15, [np.array([[190, 190], [260, 270], [130, 270]])], True, 255, 3)
cv2.putText(img_15, 'ROTATE', (15, 240), cv2.FONT_HERSHEY_SIMPLEX, 1.1, 255, 3)

# 2. Define Rotation Helper Function
def rotate_image(image, angle):
    h, w = image.shape[:2]
    center = (w // 2, h // 2)
    M = cv2.getRotationMatrix2D(center, angle, 1.0)
    return cv2.warpAffine(image, M, (w, h))

angles = [30, 60, 90, 180]
orb = cv2.ORB_create(nfeatures=500)
matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

# 3. Detect and compute features on the original image
kp_ref, des_ref = orb.detectAndCompute(img_15, None)

plt.figure(figsize=(14, 8))
print("--- Rotation Feature Matching Log ---")

# 4. Perform feature matching across all rotated image variants
for idx, angle in enumerate(angles):
    rotated_img = rotate_image(img_15, angle)
    kp_rot, des_rot = orb.detectAndCompute(rotated_img, None)

    if des_ref is not None and des_rot is not None:
        matches = matcher.match(des_ref, des_rot)
        matches = sorted(matches, key=lambda x: x.distance)

        top_matches = matches[:20]
        matched_vis = cv2.drawMatches(
            img_15, kp_ref, rotated_img, kp_rot, top_matches, None,
            flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
        )

        avg_dist = np.mean([m.distance for m in top_matches]) if top_matches else 0.0
        print(f"Angle: {angle:3d}° | Matches Found: {len(matches)} | Top Match Avg Dist: {avg_dist:.2f}")

        plt.subplot(2, 2, idx + 1)
        plt.imshow(cv2.cvtColor(matched_vis, cv2.COLOR_BGR2RGB))
        plt.title(f"Rotation: {angle}° ({len(matches)} Matches)", fontsize=10)
        plt.axis('off')

plt.tight_layout()
plt.show()
```

```
--- Rotation Feature Matching Log ---
Angle:  30° | Matches Found: 123 | Top Match Avg Dist: 8.10
Angle:  60° | Matches Found: 122 | Top Match Avg Dist: 7.95
Angle:  90° | Matches Found: 227 | Top Match Avg Dist: 0.00
Angle: 180° | Matches Found: 217 | Top Match Avg Dist: 0.00
```

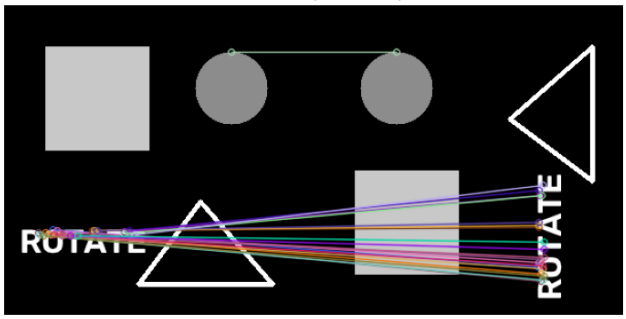
Rotation: 30° (123 Matches)



Rotation: 60° (122 Matches)



Rotation: 90° (227 Matches)



Rotation: 180° (217 Matches)